

AtCoder Beginner Contest 454 – C

Straw Millionaire の解説

問題の概要

- アイテム $1, 2, \dots, N$ の N 種類がある.
- 高橋君は最初**アイテム 1** のみを所持.
- M 人の友達がいて, i 番目の友達は「**アイテム A_i を渡すとアイテム B_i をもらえる**」という交換をしてくれる.
- 高橋君が**手に入れることのできるアイテム**は何種類あるか? (アイテム 1 を含める)

制約: $2 \leq N \leq 3 \times 10^5$, $1 \leq M \leq 3 \times 10^5$, $A_i \neq B_i$

サンプル 1 を手で動かす

$$N = 5, M = 5, \quad (A_i, B_i) = (1, 2), (2, 3), (3, 4), (2, 4), (5, 2)$$

交換の可能性を追う：

- アイテム 1 を持っている $\Rightarrow$ 友達 1 で**アイテム 2** を入手可
- アイテム 2 を持てば $\Rightarrow$ 友達 2 で**アイテム 3**, 友達 4 で**アイテム 4**
- アイテム 3 を持てば $\Rightarrow$ 友達 3 で**アイテム 4** (既にあり)
- アイテム 5 からしか出ない交換 (友達 5) は使えない (5 を持てない)

答え

到達できたアイテム: $\{1, 2, 3, 4\} \rightarrow$ **4 種類**

なぜ「グラフ」を思いつくか — 問題文の読み方

問題文の**構造だけ**を書き出してみる：

問題の本質

- 対象：アイテム（離散的， N 個）
- 関係：「 A_i を持てば B_i も手に入る」という**一方向**の条件
- 連鎖： B_i を得たら，今度は B_i を前提とする別の交換ができる
- 始点：アイテム 1
- 問い：始点から「届く」ものは何個？

この 5 つが揃ったら，**有向グラフの到達可能性問題**のサイン．

典型パターンへの翻訳

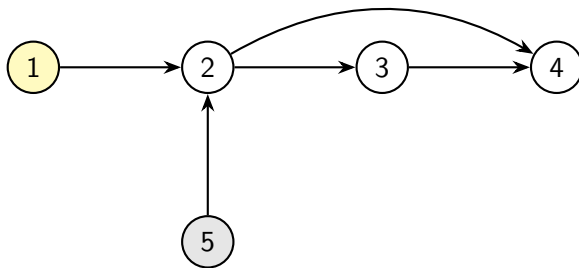
「**A** があれば **B** が得られる」型の問題は、次のように翻訳する：

問題の言葉	グラフの言葉
アイテム i	頂点 i
「 A_i を渡すと B_i がもらえる」	有向辺 $A_i \rightarrow B_i$
最初に持っているもの	始点 $s = 1$
手に入れられる種類数	s から到達可能な頂点数

思考の合言葉

「**A** $\rightarrow$ **B** の矢印が頭に浮かんだらグラフを描く」

サンプル 1 をグラフで見る



- 黄色 = 始点 1. そこから矢印で辿れる頂点は $\{1, 2, 3, 4\}$.
- 頂点 5 は始点からは到達不能.

到達可能な頂点数 = 4 = 答え

公式解説のアルゴリズム (BFS)

N 頂点・ M 辺の有向グラフで、頂点 1 から到達可能な頂点数を求める。

手順 (幅優先探索 BFS)

- 到達済み集合 S をもつ。初期値 $S = \{1\}$.
- 探索待ちのキュー q をもつ。初期値 $q = [1]$.
- q が空でない間：
 - q から 1 つ取り出して x とする.
 - x から出る辺の先 y について：
 - $y \in S$ ならスキップ.
 - そうでなければ S と q に y を入れる.
- 最後に $|S|$ が答え.

実装上は S を長さ $N + 1$ の bool 配列, q を queue で持つのが定番.

「渡すと失うのでは？」という疑問への答え

素直に読むと「 A_i を渡す」は A_i を失うように見える．が、本問では到達可能性でよい．その理由：

- 求めるのは「手に入れることのできるアイテムの種類数」．最終所持数ではなく、一度でも入手できれば良い．
- グラフで 1 から v へ経路 $1 \rightarrow u_1 \rightarrow \dots \rightarrow v$ があれば、順次交換することで途中のアイテムを全部一度ずつ入手できる．
- 到達可能な頂点集合だけが「一度でも所持しうる」候補．それ以外は絶対に入手できない．

サンプル 3 の検算（答え 6）

辺: $2 \rightarrow 6, 2 \rightarrow 5, 3 \rightarrow 6, 1 \rightarrow 6, 1 \rightarrow 2, 5 \rightarrow 6, 2 \rightarrow 3, 3 \rightarrow 7$

1 から到達可能: $\{1, 2, 3, 5, 6, 7\}$ （頂点 4 は孤立） $\Rightarrow$ **6**．

実装：C++

```
int n, m;
cin >> n >> m;
vector<vector<int>> g(n + 1);
for (int i = 0; i < m; i++) {
    int a, b; cin >> a >> b;
    g[a].push_back(b);           // A -> B の有向辺
}
```

```
vector<bool> seen(n + 1, false);
queue<int> q;
q.push(1); seen[1] = true;
int ans = 0;
while (!q.empty()) {
    int u = q.front(); q.pop();
    ans++;
    for (int v : g[u]) {
        if (!seen[v]) {
            q.push(v);
            seen[v] = true;
        }
    }
}
```

実装：PyPy

```
import sys
from collections import deque

def main():
    data = sys.stdin.buffer.read().split()
    it = iter(data)
    n = int(next(it)); m = int(next(it))
    g = [[] for _ in range(n + 1)]
    for _ in range(m):
        a = int(next(it)); b = int(next(it))
        g[a].append(b)

    seen = [False] * (n + 1)
    seen[1] = True
    q = deque([1])
```

- 辺の構築: $O(M)$
- BFS: 各頂点 1 回, 各辺 1 回訪問するので $O(N + M)$

全体で $O(N + M)$.

$N, M \leq 3 \times 10^5$ なので十分高速.

注意点

- DFS を再帰で書くと N 大でスタックオーバーフローの恐れ. BFS (キュー) か, スタックを陽に持つ反復 DFS が安全.
- 多重辺・自己ループの可能性は薄いですが, $A_i \neq B_i$ は制約で保証.

まとめ

- ① 「A を持てば B が得られる」型は有向グラフで定式化.
- ② 始点から「届くもの」を問われたら**到達可能性** → BFS/DFS.
- ③ 計算量 $O(N + M)$ で制約内.

覚えること

問題文を読んで次の 3 点が揃ったら、まずグラフを描く：
対象 × 一方向の関係 × 連鎖的な到達.